RECONNAISSANCE & FOOTPRINTING REPORT (1)

Executive Summary

Target Organization: altschoolafrica

Engagement: Passive and Active Reconnaissance

Assessment Date: April 16 , 2026

Status: Low Risk Assessment

Consultant: Chinaza Nnamani

Introduction

This report document hereby describes the proceedings and results of altschoolafrica penetration testing assessment conducted against ~~confidential~~. The report hereby lists the findings and corresponding best practice mitigation actions and recommendations.

Objective 📌

The summary of objectives are to:

- Differentiate passive vs active reconnaissance techniques.
- Perform OSINT to enumerate public-facing information about a target domain.
- Run basic active scanning using Nmap to discover hosts, open ports, and services.
- Produce a concise attack-surface Data Flow Diagram (DFD) and prioritize potential targets for further testing.

Scope 🔍

The scope of the penetration testing assessment for ~~confidential~~ includes performing Active and Passive Reconnaissance & Footprinting against ~~confidential~~. Testing will focus only on Passive Reconnaissance & Footprinting.

The boundaries of the project exclude testing of domains and subdomains that is not ~~confidential~~. A concise attack-surface Data Flow Diagram (DFD) which prioritize potential targets for further testing will be provided.

In Scope

- ~~confidential~~

Out of Scope

- Everything not in scope

Rules of Engagement

- Use only approved OSINT tools and your Kali lab (or cloud lab)
- All actions must be documented
- Only scan the authorized lab domain provided by the organization.
- Keep all findings within course submissions — do not publish or share external data.
- The scenario must be treated as a real-world engagement

💡 APPLICATION : ~~confidential~~

Passive Reconnaissance (OSINT) 📋

Action Peformed	Sources	Commands
Subdomain enumeration	Amass Recon-ng	amass enum -passive -d ~~confidential~~ # Inside the Recon-ng console: modules load recon/domains-hosts/hackertarget options set SOURCE ~~confidential~~ run
Public asset discovery	Google dorking Shodan Cloudenum	site:s3.amazonaws.com "~~confidential~~" hostname:a~~confidential~~ http.title:"Index of /" "s3.amazonaws.com" python3 cloud_enum.py -k altschoolafrica
DNS records (A, MX, TXT, CNAME) and WHOIS lookup	dig dnsrecon dnsenum	dig ~~confidential~~ dnsrecon -d ~~confidential~~ -t std dnsenum ~~confidential~~
Email addresses and possible employee names	Apollo.io theHarvester	theHarvester -d ~~confidential~~ -b duckduckgo,linkedin
Shodan lookups for known services	Shodan Web Interface Nmap shadan script	nmap --script shodan-api --script-args shodan-api.apikey=NIISpU1T5qU9Z4UUdxKXhatAyIn0nlag -sn ~~confidential~~

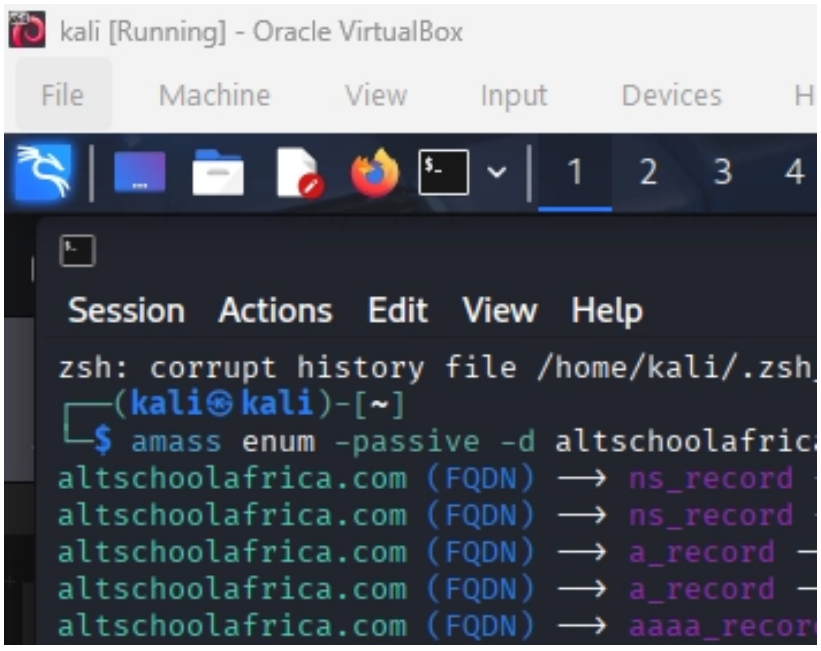
Findings	Confidence	Potential Sensitivity
Subdomains; ~~confidential~~	HIGH	PUBLIC
Public buckets; ~~confidential~~	LOW	INTERNAL
CNAME Records: ~~confidential~~	HIGH	PUBLIC
A Records (Origin IP Leaks) ~~confidential~~	HIGH	INTERNAL

Findings	Confidence	Potential Sensitivity
MX and DNSSEC Records: **Mail Routing (MX):** The MX records (e.g., `aspmx.l.google.com`) confirm the use of Google Workspace to handle corporate email. **Lack of DNSSEC:** The `dnsrecon` tool explicitly noted: `[-] DNSSEC is not configured for` ~~confidential~~. This means the DNS responses are not cryptographically signed, technically leaving them open to DNS spoofing/cache poisoning attacks.	HIGH	PUBLIC
Staging Environments: Several development and staging environments were exposed, which often have weaker security controls or debugging enabled: ~~confidential~~	MEDIUM	INTERNAL
Administrative Panels: The brute-force enumeration via **dnsenum** discovered ~~confidential~~. Adminer is a well-known, single-file database management tool (similar to phpMyAdmin), making this a highly critical target for further testing.	LOW	INTERNAL
Discovered emails; ~~confidential~~	HIGH	INTERNAL

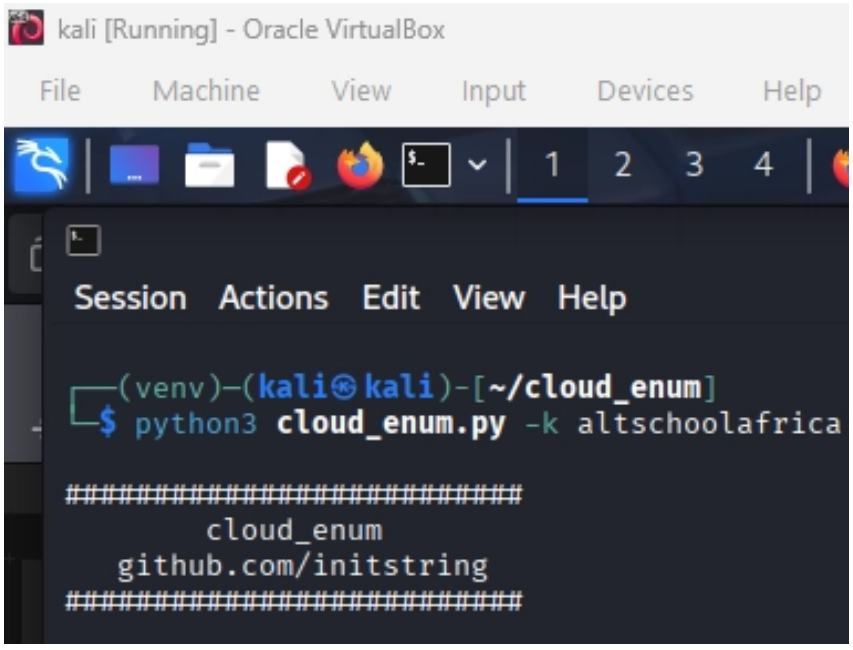
Findings

PROOF OF CONCEPT 🧾

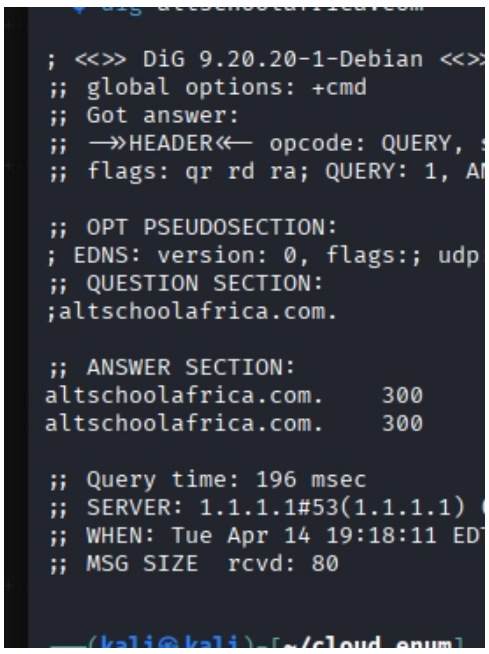
Subdomain enumeration (e.g., Amass, Recon-ng).

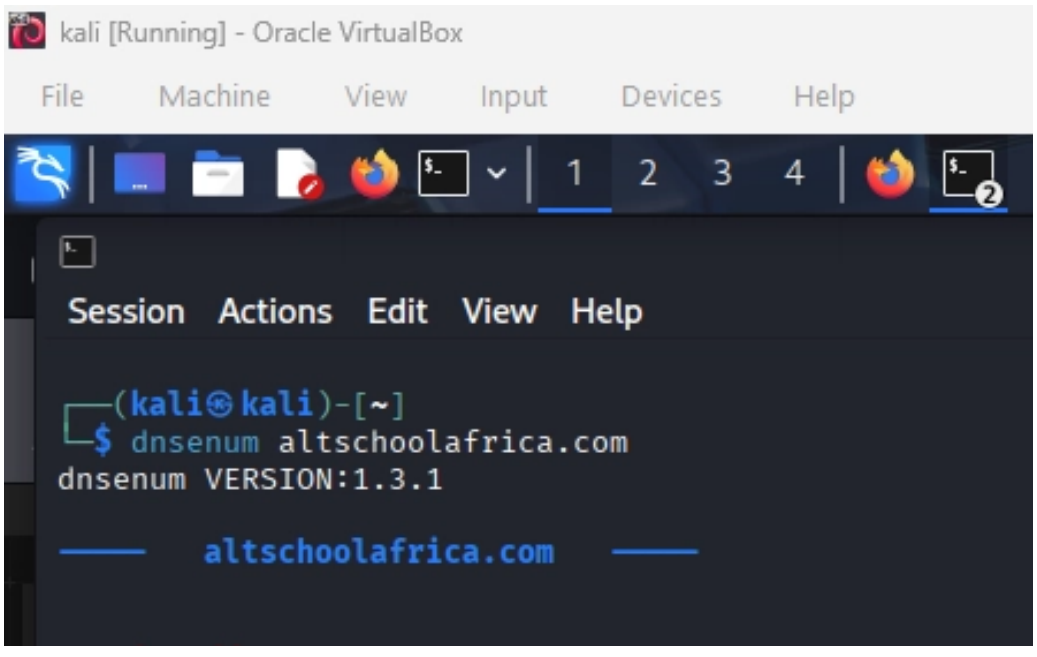


Public asset discovery (S3 buckets, exposed files).

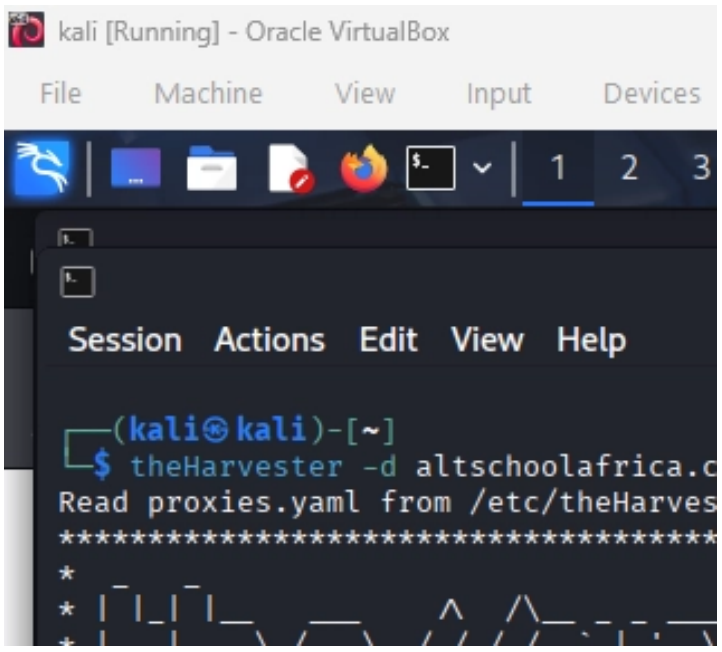


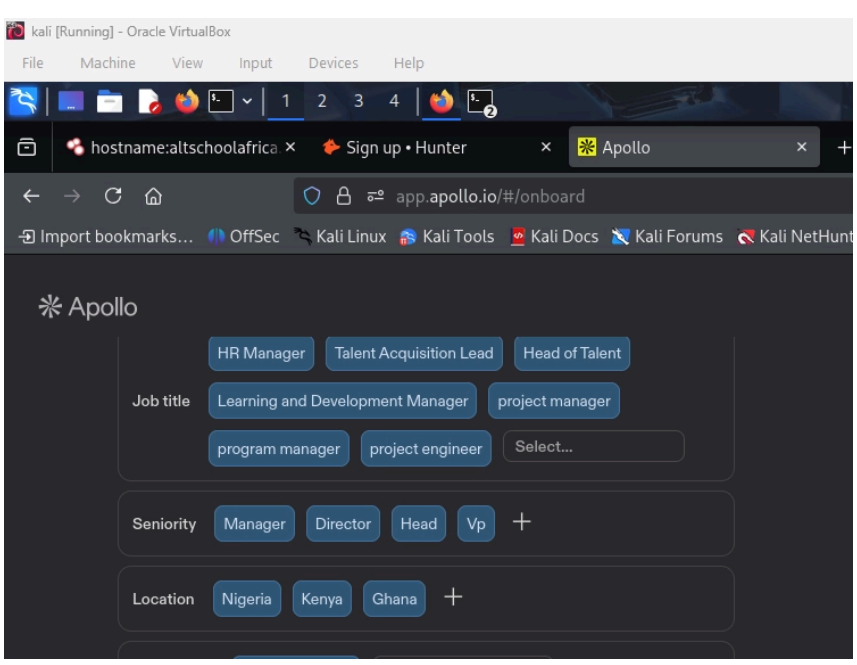
DNS records (A, MX, TXT, CNAME) and WHOIS lookup.



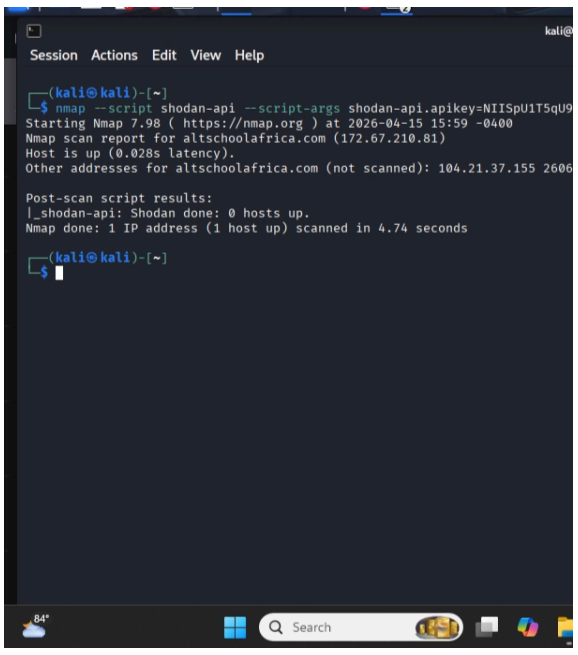


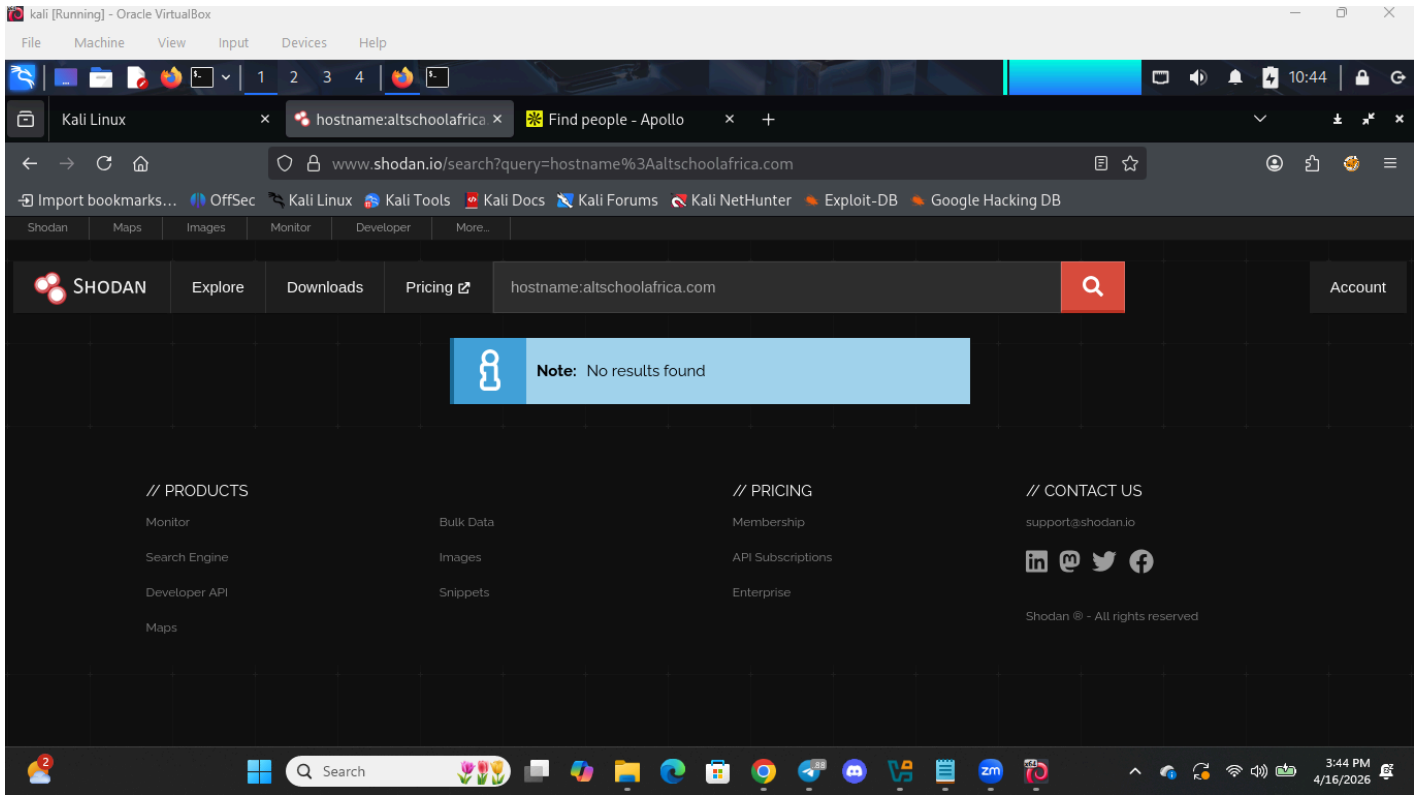
Email addresses and possible employee names





Shodan lookups for known services (if any).





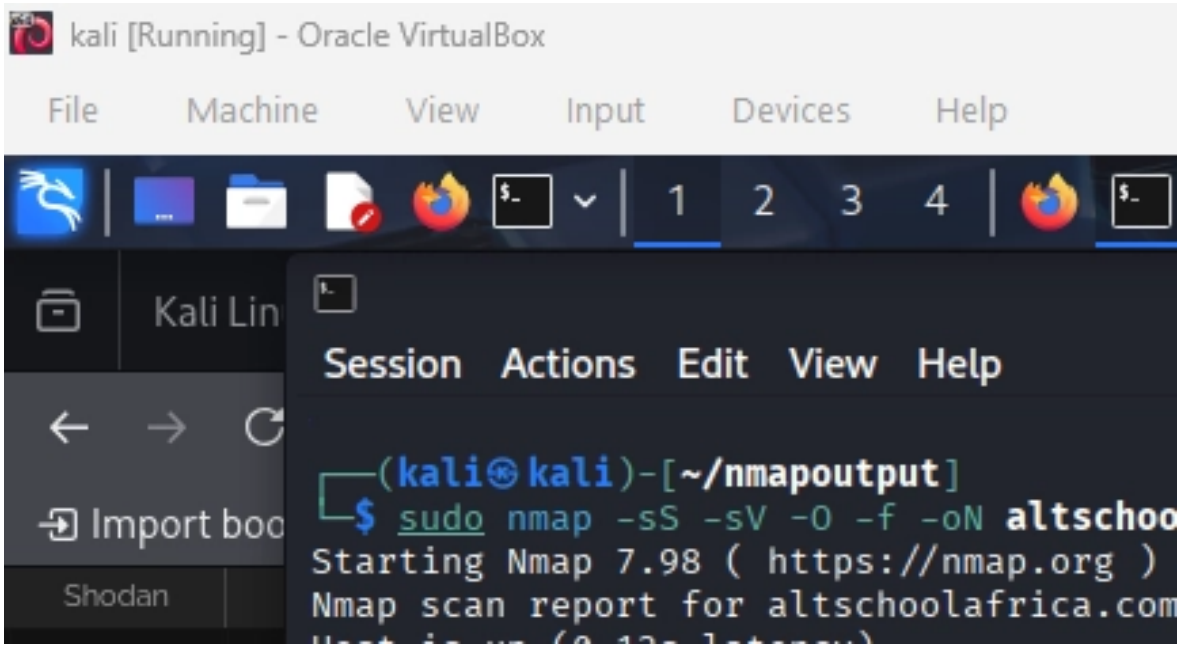
Active Scanning (Nmap) 🔎

💡 Discovered Host IPs; ~~confidential~~

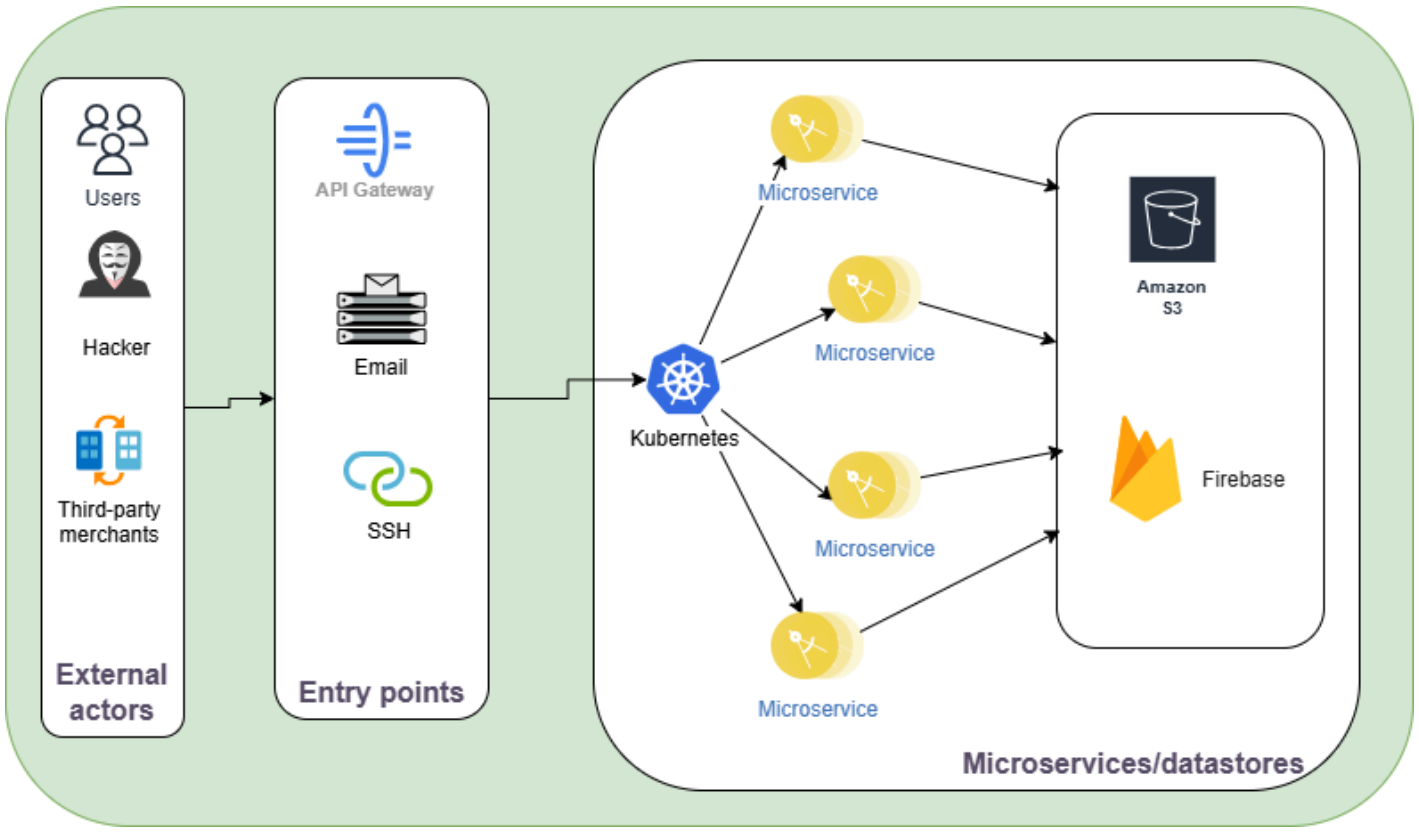
Actions Performed	Results
Host discovery scan `nmap -sn -oN nmap_recon.txt` ~~confidential~~	This was a simple ping sweep or host discovery scan (`sn`). It did not scan any ports. It simply sent ICMP and TCP probes to check if the target (~~confidential~~) was online, which it confirmed it was.
Service/version detection `nmap -sC -sV -p- -oN nmap_recon1.txt` ~~confidential~~	It scanned all 65,535 possible TCP ports. It also used default scripts (`sC`) and version detection (`sV`). This scan successfully discovered several non-standard ports that Cloudflare leaves open for specific web/proxy traffic (like 2052, 2053, 2082, etc.), identifying them as either Cloudflare proxies or Nginx servers.
Aggressive scan `nmap -A -oN nmap_recon2.txt` ~~confidential~~	This was an "Aggressive" scan (`A`), which automatically enables OS detection, version detection, script scanning, and traceroute. Scanning IP ~~confidential~~, it identified five open ports and confirmed that ports 80, 443, 8080, and 8443 are running Cloudflare's HTTP proxy service. It also mapped a 5-hop traceroute to the Cloudflare edge server.
TCP SYN stealth scan `nmap -sS -oN altschool_nmap2.txt` ~~confidential~~	This was a basic TCP SYN "stealth" scan (`sS`) targeting the default top 1,000 ports on the IP ~~confidential~~. It successfully identified five open ports (80, 443, 587, 8080, 8443) but did not probe them to determine the specific service versions. 995 ports were filtered, meaning a firewall dropped the packets without responding. The stealth scan was not able to evade the firewall.
Fragmented TCP SYN stealth scan `nmap -sS -sV -O -f -oN` ~~confidential~~	In this scan, an attempt was made to evade the firewall by fragmenting packets (`f`) while simultaneously asking Nmap to determine service versions (`sV`) and guess the Operating System (`O`). The fragmentation evasion failed completely; the Cloudflare firewall blocked all 1,000 scanned ports (showing them as filtered). Because no open ports were found, Nmap's OS detection algorithm guessed wildly, inaccurately suggesting Cisco, APC, or Synology devices.

Open Port	Service	Version
80	http	Cloudflare http proxy
443	ssl/http	Cloudflare http proxy
587	submission?	Unknown (Connection failed)
2052	http	Cloudflare http proxy
2053	ssl/http	nginx
2082	http	Cloudflare http proxy
2083	ssl/http	nginx
2086	http	Cloudflare http proxy
2087	ssl/http	nginx
2095	http	Cloudflare http proxy
2096	ssl/http	nginx
8080	http	Cloudflare http proxy
8443	ssl/http	Cloudflare http proxy
8880	http	Cloudflare http proxy

PROOF OF CONCEPT 🧾



Attack Surface DFD & Prioritization



Based on the reconnaissance data, specifically the `cloud_enum`, `dnsenum`, and `recon-ng` outputs, we can deduce a significant amount about the target's backend architecture. The naming conventions of their cloud assets and subdomains strongly indicate a modern, cloud-native infrastructure.

Here are the datastores and microservices identified:

1. Datastores (Storage & Databases)

- **Google Firebase Realtime Databases (NoSQL):** The `cloud_enum` scan revealed several Firebase databases, which are often used as backend datastores for real-time applications.

  ~~confidential~~

- **AWS S3 Buckets (Object/Blob Storage):** Two specific buckets were identified, suggesting data storage patterns.

  ~~confidential~~: The naming convention strongly implies this is used for internal data storage or backups.
  ~~confidential~~: The "-es" suffix often denotes an Elasticsearch snapshot repository or a cluster backing store, pointing to the use of Elasticsearch for search or logging.
  Relational Database UI: * ~~confidential~~: The presence of Adminer, a database management tool, confirms they are running a relational database (like MySQL or PostgreSQL) internally.

2. Microservices & Compute Architecture

- **Container Orchestration (Kubernetes):** Google Cloud Functions were found with names explicitly referencing Kubernetes, highly suggesting their core microservices are containerized.

  ~~confidential~~

- **Serverless Functions:** The organization relies on Google Cloud Functions for event-driven microservices.
  ~~confidential~~
- **Data Pipelines:** * ~~confidential~~: "Dataflow" suggests they use Google Cloud Dataflow for stream or batch data processing pipelines.
- **Domain-Driven Service Routing (Inferred from DNS):** Several subdomains hint at a microservice architecture where different application functions are segmented into their own environments or APIs.
  ~~confidential~~ (Likely the core API/backend service)
  ~~confidential~~ (Often used for atomic components or a specific internal service)
  ~~confidential~~ (Likely an integration or API gateway service)
  ~~confidential~~ (Likely serving or analyzing data)

Top 5 potential targets based on the reconnaissance data

1. The ~~confidential~~Subdomain Origin IPs

- **Assets:** ~~confidential~~
- **Risk Statement:** These IPs completely bypass the Cloudflare WAF, exposing the true AWS backend infrastructure. This allows for direct, unmitigated active scanning and exploitation attempts without triggering edge-layer defenses or rate limiting.

2. Adminer Interface

- **Asset:** ~~confidential~~
- **Risk Statement:** Exposes a web-based relational database management tool directly to the internet. If it is running an outdated version with known CVEs, or if it is protected by weak/default credentials, it provides a direct vector to total database compromise and potential Remote Code Execution (RCE).

3. Cloud Storage & Firebase Databases

- **Assets:** ~~confidential~~
- **Risk Statement:** Cloud datastores are prone to access control misconfigurations. If any of these S3 buckets or Firebase endpoints allow unauthenticated read or write access, it results in an immediate, zero-click breach of sensitive data or application logic.

4. Staging & Development Environments

- **Assets:** ~~confidential~~
- **Risk Statement:** Staging environments frequently run with verbose debugging enabled, utilize weaker testing credentials, and lack production-grade security monitoring. They often serve as the path of least resistance for establishing an initial foothold in the network.

5. Email Infrastructure

- **Asset:** DMARC Record (`v=DMARC1; p=none`)
- **Risk Statement:** The permissive `p=none` DMARC policy allows malicious actors to send highly convincing, spoofed emails that appear to legitimately originate from ~~confidential.~~ This creates a high-probability vector for targeted phishing campaigns aimed at credential harvesting or payload delivery.